vibe-debug

Universal AI pipeline observability for vibe coders

What It Does

vibe-debug is a local-first debugging toolkit for Python pipelines.

You add one line to your project:

```
import vibe_debug
vibe_debug.start("my-project")
```

From that point on, *vibe-debug* can:

- trace function calls automatically
- capture crashes and silent failures
- attach file paths and line numbers to spans
- intercept outbound HTTP and TCP calls
- build a pipeline map from static code analysis before your code runs
- store everything in a local Phoenix dashboard
- expose answers back to your IDE assistant through MCP tools

The goal is simple: when something breaks, your IDE can tell you what failed, where it failed, what never ran, and what the rest of the project looks like.

Quick Start

```
pip install arize-phoenix opentelemetry-sdk opentelemetry-exporter-otlp-proto-http mcp httpx wrapt
```

```
# Terminal 1
python -m phoenix.server.main serve
```

```
# Terminal 2
python -m server.mcp_bridge
```

```
# In your project - add ONE line
import vibe_debug
```

```
vibe_debug.start("my-project")
```

What You Get Automatically

- Every function in watched modules traced with zero decorators
- Exact file path and line number on every span
- Silent failure detection for empty returns such as *None*, *[]*, *""*, and *{}*
- Exception capture with runtime context
- Every HTTP and TCP call captured at runtime
- Auto-generated *vibedebug.toml* from static analysis
- A pipeline map built before the project runs a single line
- Crash impact analysis showing what never ran and why
- IDE integration through MCP tools such as *get_last_failure()*

Manual Decoration (Optional)

```
from vibe_debug.decorators import vibetrace


@vibetrace(component_type="rag")
def retrieve_documents(query: str):
    """Fetches relevant documents from vector store."""
    return faiss_index.search(query)
```

IDE Integration

Run the MCP bridge and connect your IDE assistant to it.

Available tools:

- *get_last_failure()* - exact component, file, line, and status from the latest failure
- *explain_trace()* - short waterfall summary of the latest trace
- *get_pipeline_map()* - current discovered project structure, crash history, and never-observed components

Project Structure

```
vibe-debug/
■■■ vibe_debug/
■   ■■■ __init__.py                  ← one-line start() entry point
```

```
■   ■■■ tracer.py                    ← OpenTelemetry tracer/export setup
■   ■■■ decorators.py                ← optional @vibetrace decorator
■   ■■■ auto_instrument.py           ← wrapt-based zero-decorator patching
■   ■■■ adapters.py                  ← placeholder adapter hook
■   ■■■ config/
■   ■   ■■■ __init__.py              ← config exports
■   ■   ■■■ config.py                ← vibedebug.toml reader
■   ■   ■■■ config_generator.py      ← auto-generates vibedebug.toml
■   ■■■ runtime/
■       ■■■ __init__.py              ← runtime package marker
■       ■■■ static_analyzer.py       ← AST-based project scanner
■       ■■■ import_mapper.py         ← import dependency graph builder
■       ■■■ partial_trace_builder.py ← crash + NEVER_REACHED assembly
■       ■■■ pipeline_discovery.py    ← unified pipeline map cache
■       ■■■ sys_tracer.py            ← universal sys.settrace() hook
■       ■■■ io_interceptor.py        ← HTTP/socket interception
■■■ server/
■   ■■■ mcp_bridge.py                ← MCP server for IDE integration
■   ■■■ manager.py                   ← Phoenix server launcher
■■■ tests/                           ← integration, runtime, and analysis tests
■■■ vibedebug.toml                   ← generated/configured watch list
■■■ .vibe_pipeline_cache.json        ← persisted discovered pipeline map
```

Configuration (`vibedebug.toml`)

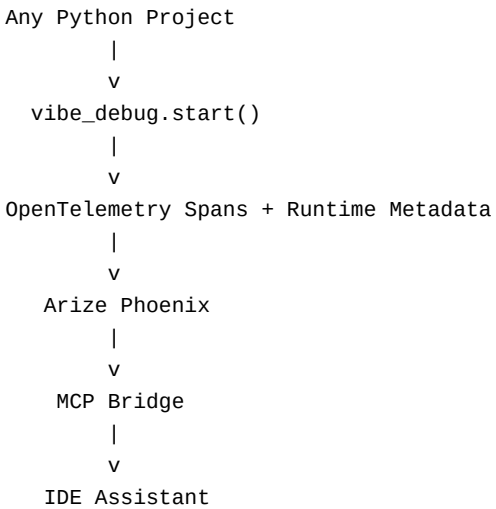
Example:

```
[project]
name = "vibe-debug"

[tracer]
otlp_endpoint = "http://localhost:6006/v1/traces"

[watch]
modules = ["tests.mock_pipeline"]
ignore_functions = ["__init__", "__repr__", "__str__"]

[discovered_components]
total_functions = 119
total_modules = 25
entry_points = [
  "unknown - no main.py or app.py detected"
]

[thresholds]
slow_call_ms = 500
silent_failure_types = ["rag", "router", "agent", "knowledge_graph"]

[adapters]
enabled = ["ollama", "langchain", "openai"]
```

What each section means:

- *[project]* - project name used in traces and reports
- *[tracer]* - OTLP destination, usually Phoenix on *localhost:6006*
- *[watch]* - modules to patch automatically and functions to ignore
- *[discovered_components]* - metadata generated from static analysis
- *[thresholds]* - latency and silent-failure defaults
- *[adapters]* - reserved adapter config area

Requirements

- Python 3.11+
- Phoenix running locally for storage and UI
- Main packages used in this repo:
- *arize-phoenix*
- *opentelemetry-sdk*
- *opentelemetry-exporter-otlp*
- *mcp[cli]*
- *httpx*
- *pytest*
- *python-dotenv*
- *fastmcp*
- *wrapt*

Architecture Diagram